the window and examine its contents to analyze it (for example, to check the length of the session). You may never get an event to close the session, but you can also use a time limit to close the window. You can build this logic into the actor that a stream uses.

To learn how streams and actors work together, let's look at a much simpler case, stripped down to the basics: an actor implementing a counter that responds to the stream with the counter value. This counter has a limit (it is capped). It could be used to limit calling an external service, as a kind of throttle. This actor is shown in the following listing.

Listing 16.3 Counter with a `max` value

```
object Cap {

    case class Increment(increment: Int, replyTo: ActorRef[Int])     ❶

    def apply(current: Int, max: Int): Behavior[Increment] = {       ❷
      Behaviors.receiveMessage { message =>
        message match {
          case Increment(increment, replyTo) =>
            if (current + increment > max) {                         ❸
              replyTo ! current
              Behaviors.same
            } else {
              replyTo ! current + increment
              apply(current + increment, max)
            }
    }}}
}
```

❶ Protocol

❷ Initialization and state of the actor

❸ Logic to avoid surpassing the cap

This actor responds with the `current` value, increment included, on each `Increment` requested message. The value of `max` is set to